Product Requirements

Document Map

1. **Product definition** — mission, users, and core jobs.
2. **Scientific and product workflows** — intake, formulation, characterization, candidates, laboratory feedback, and evolutionary campaigns.
3. **Intelligence, reasoning, and optimization** — LLM integration, neuro-symbolic reasoning, evolutionary search, and cross-project learning.
4. **Knowledge and data platform** — refinery, scientific library, evidence promotion, and reusable knowledge.
5. **Extensible capability-node platform** — capability discovery, project matching, node creation, validation, registry, composition, and lifecycle.
6. **Enterprise platform requirements** — identity, integration, deployment, security, reliability, commercial controls, and adoption.
7. **User experience and role workspaces** — Operator/DevOps, external customer, data scientist, agent management, customer, and experiment slices.
8. **Cross-domain auditability** — immutable events, reconstruction, traceability, exports, privacy, and failure posture.
9. **Agent-orchestrated environment** — durable work, bounded autonomy, human control, tools, support, and agent lifecycle.
10. **Boundaries and acceptance** — non-goals and product acceptance posture.

Product Mission

Build a materials R&D operating system for coatings and adjacent advanced materials. The system helps scientists and program owners turn a partner's manufacturing envelope into auditable candidate formulations, lab validation plans, measured evidence, and eventually manufacturing-release packages.

The product promise is not "generate molecules." The promise is "advanced materials, made manufacturable": every candidate must be designed around plant constraints, feedstocks, cost ceilings, process route, qualification path, and evidence limits from the first pass.

Primary Users

- Formulation scientists who define ingredients, compose formulations, review computed properties, and plan experiments.
- Program leads who define target product profiles, phase gates, cost ceilings, lab plans, and customer deliverables.
- Lab/CRO operators who receive formulation/process packets and return measurements, deviations, and custody records.
- Evidence reviewers who inspect source rights, model authority, claim locks, audit receipts, and qualification limits.
- Knowledge curators and refinery operators who review candidate sources, normalized claims, retrieval material, evaluations, promotions, and rollbacks.
- Business operators who need partner-ready summaries without overstating authority.
- Capability/node developers who implement, test, package, document, and maintain specialist methods and integrations.
- Scientific capability reviewers who determine node applicability, validation posture, decision eligibility, and domain-pack membership.

- Platform operators/DevOps engineers who deploy, observe, secure, recover, and support the service and its agent/node fleet.
- Customer and enterprise administrators who manage federation, users, groups, policies, entitlements, integrations, data posture, and access reviews.
- External customer participants who provide requirements, resolve ambiguities, approve customer-owned decisions, monitor work, and receive authorized deliverables.

Core Jobs To Be Done

1. Convert an unstructured partner ask or formal technical envelope into a signed `ProductEnvelope`.
2. Maintain an ingredient/feedstock library with costs, suppliers, constants, restrictions, and provenance.
3. Compose and clone immutable formulation versions with parent-child lineage.
4. Compute coating physics and other domain-pack properties from composition and ingredient constants.
5. Generate candidates only inside the manufacturing envelope.
6. Route candidate checks through specialist modules that may pass, fail, abstain, or mark not applicable.
7. Assemble auditable candidate packages with process route, sourcing note, cost estimate, and authority status.
8. Create lab/CRO protocols and test matrices from the envelope.
9. Ingest measured lab results with custody, raw files, methods, calibration, and deviations.
10. Use accepted empirical returns to improve models and select the next experiment batch.
11. Produce a manufacturing-release package when evidence supports it: locked composition, process window, QC/SPC release criteria, and supplier-qualified BOM.
12. Run bounded evolutionary campaigns that preserve candidate lineage, mutation/recombination operators, retained rejects, retained duplicates, fitness evidence, and selection decisions.
13. Refine retrieved, curated, measured, and model-distilled material into governed scientific knowledge without allowing unreviewed candidate knowledge to alter live experiments or trusted stores.
14. Convert LLM prompt outputs into typed, reviewable contracts before they can drive candidate generation, fitness, selection, reports, or lab packages.
15. Compile each project envelope into required capabilities, match eligible node versions, expose gaps, and approve an immutable capability plan and execution lockfile.
16. Build, evaluate, package, deploy, observe, promote, quarantine, migrate, and retire new specialist nodes without changing the orchestrator core.
17. Operate multiple enterprise customers with federated identity, tenant isolation, measurable service objectives, supported integrations/deployments, cost governance, and verifiable auditability.
18. Orchestrate authorized work through agents and nodes while retaining accountable human approvals, recoverable execution state, complete provenance, and safe escalation.

Scientific and Product Workflows

Envelope Intake

Inputs may be a technical envelope, deck, email, one-page ask, or verbal brief. The system must produce structured constraints with:

- requirement text
- hard gate, soft objective, or information-only classification
- units
- test method anchor where available
- provenance: `CUSTOMER_STATED`, `INFERRED`, or `DEFAULTED`
- authority and sign-off status
- ambiguity register for missing information

Envelope sign-off blocks expensive candidate generation.

The UI must support initial human feedback during this stage. Users need a review surface where they can confirm, correct, reject, or annotate extracted constraints, assumptions, units, test methods, priorities, restricted substances, process limits, cost ceilings, and ambiguity-register items before a `ProductEnvelope` is treated as reviewed.

Formulation Design

The MVP must support coatings formulation design:

- ingredient library
- formulation components by mass fraction
- live computed properties
- clone/version flow for iteration
- experiment records and measurements
- suggestion engine for next formulations

Incumbent Characterization and Baseline Reconstruction

For reverse-engineering programs such as the initial naval paint effort, characterization must precede candidate optimization. The product must not represent a cured coating as one recoverable molecular structure or imply that analytical evidence uniquely determines the original recipe. It must produce a governed composition and process fingerprint containing quantitative ranges, confidence, competing hypotheses, method limitations, mass balance, and an explicit unidentified fraction.

The workflow must support:

- authorization, data-rights, safety, specification/detail-sheet, and chain-of-custody intake;
- wet Part A/Part B, mixed, cured, aged, and service-exposed sample states where available;
- samples, aliquots, fractions, preparation transformations, standards, blanks, controls, replicates, and deviations;
- methods, method versions, instruments, calibration, raw-file hashes, detection/quantitation limits, units, uncertainty, and reviewer disposition;
- volatile, soluble-organic, insoluble inorganic, cured-network, bulk-property, and applied-film observations without assuming every method applies to every sample;
- ingredient/component hypotheses with ranges, alternatives, supporting and conflicting observations, confidence, and review state;
- separate and combined mass balances with unresolved peaks, phases, contradictions, and unidentified fractions preserved;
- method-aware analytical and process fingerprints and a recommendation for the next measurement that would reduce consequential uncertainty;
- versioned baseline-reconstruction formulations, preparation procedures, incumbent comparisons, and explicit accept/reject decisions.

If only cured material is available, the campaign must declare an underdetermined-reconstruction posture and limit its claims accordingly. Optimization remains blocked until the program's scientific reviewers accept at least one baseline reconstruction against predeclared chemical, processing, cure, and relevant performance gates, or explicitly authorize a different evidence-based starting posture.

Neuro-symbolic reasoning may propose identities and reconcile spectral, elemental, stoichiometric, mass-balance, process, knowledge, and authority evidence while retaining alternatives and contradictions. LLM or statistical matches remain hypotheses until the required deterministic and human review occurs. Genetic/evolutionary search begins only after the reconstruction and `ProductEnvelope` gates; it varies authorized formulation dimensions and must not be used to fabricate the original recipe from incomplete characterization evidence.

The current detailed implementation assessment is maintained in `Z:\mvp20-split\docs\MVP-FEATURE-GAP-ANALYSIS.md`.

Candidate Compilation

Candidate output must include:

- formulation identity and lineage
- component list
- computed properties
- envelope gate results
- process route or recipe
- sourcing note and cost estimate
- evidence status per module
- limitations and abstentions
- lab handoff instructions

Lab-In-The-Loop Optimization

The system must treat wet-lab results as first-class evidence, not notes. Lab returns must be candidate-bound and eligible for model promotion only after validation, custody, and authority checks.

Evolutionary Candidate Campaigns

The preferred optimizer workflow is a genetic-programming style campaign. The system must support seeded candidate populations, formulation or structure variation operators, deterministic replay, typed fitness components, and policy-authorized selection.

Early campaigns may use deterministic enumeration with ranking disabled. As measured evidence arrives, the system may enable active-learning or genetic-selection strategies behind the same campaign model.

Intelligence, Reasoning, and Optimization

LLM Prompt Integration

The LLM prompt layer must be an intake, translation, planning, and explanation surface. It must not be the hidden optimizer.

The required integration contract is:

```
prompt material -> ProductEnvelope
ProductEnvelope -> EvolutionCampaign
EvolutionCampaign -> CandidateAttempts + FitnessComponents
PolicySolver -> SelectionDecision
SelectionDecision -> LLM report or CRO packet
LabResults -> EvidenceIntake -> next EvolutionCampaign generation
```

LLM outputs may:

- extract partner intent, constraints, objectives, standards, methods, assumptions, and ambiguities from source material;
- draft clarification questions and ambiguity registers;
- propose a structured `ProductEnvelope` for review;
- propose seed candidates, allowed variables, possible operators, lab plans, and report language;

- explain candidate decisions, failed gates, evidence gaps, abstentions, and limitations;
- assemble customer-facing or CRO-facing packets from typed system records.

LLM outputs must not:

- create approved constraints without sign-off;
- mutate the signed envelope or authority policy;
- act as an untyped fitness function;
- assign scientific authority to a model, source, or candidate;
- hide whether a claim came from deterministic calculation, empirical evidence, retrieved literature, policy, or model-generated text.

The genetic/evolutionary engine consumes structured contracts, not prose. For the naval paint use case, the charter and technical envelope should compile into a machine-readable envelope containing chemistry, hard constraints, soft objectives, open formulation variables, qualification references, and authority limits. The evolutionary campaign then searches inside that contract.

The UI must make this compilation inspectable. A user should be able to compare the original prompt/source text, the LLM-extracted structured fields, unresolved ambiguities, and the compiled campaign contract before approving generation. Feedback captured here becomes typed review input, not free-form hidden prompt context.

Neuro-Symbolic Reasoning and Evolutionary Search

The product must treat neuro-symbolic reasoning and genetic/evolutionary search as complementary but distinct subsystems. They must share typed contracts and lineage without collapsing language-model reasoning, deterministic rules, and search fitness into one opaque score.

Neuro-symbolic uses

The neuro-symbolic layer combines probabilistic language/model assistance with deterministic domain objects, ontologies, equations, constraints, authority policies, and evidence graphs. It should be used to:

- extract candidate requirements, units, methods, entities, relationships, assumptions, and ambiguities from unstructured partner material;
- map extracted concepts into a reviewable `ProductEnvelope`, ingredient taxonomy, process ontology, qualification plan, and evidence-claim graph;
- normalize scientific language and propose claim records while preserving exact source passages, provenance, confidence, scope, freshness, and contradictions;
- resolve synonyms and entity identities across ingredients, suppliers, grades, test methods, standards, formulations, samples, and measured endpoints;
- reason over symbolic plant, feedstock, cost, safety, process, compatibility, test-method, authority, and qualification constraints;
- select and compose deterministic calculators or specialist modules based on declared applicability, while allowing `PASS`, `FAIL`, `ABSTAIN`, and `NOT_APPLICABLE` outcomes;
- explain why a candidate passed, failed, abstained, or was excluded using the exact rules, facts, source claims, and calculations involved;
- identify missing knowledge and propose bounded retrieval, measurement, or clarification actions;
- compare retrieved claims, measured evidence, and deterministic predictions without treating model-generated prose as scientific fact;
- compile a signed, machine-executable search contract for downstream enumeration, active learning, or evolutionary optimization.

Language or foundation models may assist extraction, semantic matching, hypothesis generation, retrieval planning, and explanation. They must not silently create approved constraints, measured facts, supplier qualifications, scientific authority, or fitness values. Every model-derived assertion remains a candidate until deterministic validation and the required human or external-authority review occur.

Genetic and evolutionary uses

The genetic/evolutionary layer explores a bounded, already-authorized design space. It should be used to:

- seed formulation or molecular populations from approved ingredients, parent candidates, reviewed hypotheses, and accepted empirical returns;
- apply typed, versioned, policy-authorized mutation and recombination operators;
- reject or repair infeasible offspring at operator time rather than generating outside the manufacturing envelope and filtering only at the end;
- calculate typed fitness components only from approved deterministic arithmetic, authorized models, accepted measured evidence, and declared soft objectives;
- preserve full parentage, generation, random seed, operator identity/version, changed fields, retained duplicates, retained rejects, constraint failures, fitness evidence, and selection decisions;
- maintain diversity and explicit exploration/exploitation posture without allowing novelty alone to override hard constraints or authority limits;
- support deterministic replay and like-for-like comparison of enumeration, active-learning, expert-selected, and evolutionary strategies;
- stop, abstain, or fall back to enumeration when operators, evidence, or fitness modules lack authority.

Required interaction boundary

The neuro-symbolic layer defines and explains **what is allowed, known, uncertain, and required**. The genetic/evolutionary layer searches **within that compiled contract**. Neuro-symbolic output may propose seeds, operators, constraints, objectives, or experiments, but only reviewed symbolic contracts may govern a live campaign. Evolutionary outcomes return as candidate evidence for neuro-symbolic explanation and human review; they do not promote their own rules, knowledge, or authority.

The system must never:

- use an LLM's preference or prose judgment as an untyped fitness function;
- allow a genetic operator to mutate the signed envelope, authority policy, evidence lineage, test definitions, or hard manufacturing constraints;
- convert a high evolutionary score into a claim of qualification, safety, procurement suitability, or measured performance;
- hide which portion of a decision came from symbolic rules, deterministic calculations, empirical models, retrieved evidence, or stochastic search.

Cross-project learning and reusable experience

The product must distinguish campaign evolution, cumulative knowledge, statistical model learning, and optimizer experience. Preserving project history or improving a population during one campaign does not by itself constitute learning across projects.

Cross-project reuse must be implemented through four separately governed surfaces:

1. **Project evidence stores** retain customer/project formulations, samples, measurements, raw files, decisions, and restricted context. Project evidence is private by default and cannot enter shared learning merely because it exists.

2. **Reviewed shared knowledge** contains rights-eligible, promoted ingredient identities, synonyms, analytical signatures, scientific claims, compatibility and stoichiometric rules, test-method knowledge, known contradictions, and bounded applicability metadata.
3. **Model registry** contains versioned statistical or learned models with exact training data/snapshot lineage, domain and endpoint scope, validation results, uncertainty behavior, known limitations, approval state, and rollback boundary.
4. **Campaign experience store** contains rights-eligible candidate outcomes, reusable seeds or motifs, operator performance, infeasibility patterns, explored-region summaries, and selection-policy evaluations without silently exposing customer formulations or objectives.

The neuro-symbolic subsystem may improve across projects by consuming promoted shared knowledge and reviewed rules. That improvement is governed knowledge accumulation, not autonomous conversion of model output into fact. It must retain the source, evidence type, scope, exceptions, contradictions, promotion decision, and knowledge-pack version used in every inference.

Statistical models may improve from pooled accepted measurements only through an explicit pipeline:

```
accepted project evidence -> rights/privacy and compatibility review -> canonical domain dataset -> training -> independent validation -> candidate model -> shadow/limited-active evaluation -> reviewed promotion
```

Cross-project training and retrieval must enforce customer authorization, data rights, confidentiality, de-identification where applicable, purpose limitations, deletion/retirement obligations, and protections against reconstruction or leakage of customer-specific formulations. A project must be able to prohibit its evidence from entering shared knowledge, training, seeds, operator statistics, or reports while still using project-local learning.

Transfer must be applicability-aware. The system must not pool epoxy, polyurethane, acrylic, silicone, antifouling, or other coating families as though they share identical chemistry or failure mechanisms. Every reusable claim, rule, model, seed, and operator statistic requires chemistry-family, formulation-space, substrate, process, test-method, endpoint, exposure, and authority scope as applicable. Out-of-domain use must abstain, remain advisory, or require an explicitly validated adaptation.

Genetic/evolutionary campaigns may reuse approved baselines, formulation motifs, surrogate models, explored-region summaries, and operator success/failure statistics from compatible prior projects. Historical success is a prior or proposal aid, not fitness evidence for the new project, and it cannot override the new project's signed envelope, hard constraints, accepted measurements, or authority policy. The system must record exactly which prior artifacts influenced population seeding, operator choice, surrogate prediction, or selection.

The initial cross-project learning implementation should prefer hierarchical reuse:

- global materials vocabulary and stable scientific knowledge;
- coatings-wide representations and test-method knowledge;
- chemistry-family and endpoint-specific rules/models;
- customer-authorized reusable project evidence;
- project-local adaptations and observations.

Promotion, rollback, and regression evaluation must operate independently at each layer so that a faulty shared rule, model, or campaign prior can be retired without rewriting retained project evidence or corrupting unrelated chemistry families.

Knowledge and Data Platform

Knowledge Refinery

The system must include a bounded background knowledge-refinery workflow that converts candidate material into reviewed, reusable knowledge. Live experiment and candidate services consume stable approved knowledge; refinery workers operate on candidate copies; promotion gates decide what becomes trusted.

The required stage boundary is:

```
raw source -> normalized record -> candidate claim/signal -> reviewed signal -> retrieval/training export -> model or adapter build
```

Each stage must be rebuildable from its upstream artifacts. A rejected, stale, superseded, or rights-blocked item must be removable by selecting a prior accepted boundary or rebuilding forward from a changed reviewed set; the system must not rely on subtracting knowledge from an already-trained model.

The refinery must:

- ingest queued web, literature, supplier, standards, patent, laboratory, operator-curated, and approved model-distillation inputs;
- retain raw source identity and immutable content hashes while keeping raw material separate from trusted retrieval stores;
- normalize, deduplicate, classify, and trust-score sources and extracted claims;
- detect changed sources, contradictory claims, stale evidence, duplicate evidence, and scope conflicts instead of flattening them into one answer;
- create evidence-backed claim objects containing the source document, bounded evidence snippet or fact window, normalized claim, confidence, scope tags, freshness/revalidation posture, rights status, and full refinery lineage;
- distinguish ordinary retrieval chunks from reviewed evidence-backed claims so downstream users and models can separate recommendation, reasoning, available evidence, and uncertainty;
- maintain explicit `candidate`, `stable`, and `journal` layers, with promotion decisions and rollback metadata preserved in the journal;
- support `pending`, `approved`, `deferred`, `rejected`, `superseded`, and `retired` governance states;
- run bounded evaluations before promotion and post-promotion regression checks against known-answer, negative-control, authority, privacy, and scientific-claim-lock suites;
- support shadow or limited-active evaluation for changes that can affect candidate generation, fitness, routing, scientific claims, or experiment selection;
- create immutable promotion packets naming the candidate inputs, target knowledge surfaces, evaluations, approval identity, expected effects, validation checks, and rollback boundary;
- prevent refinery workers from mutating live trusted stores directly; promotion must occur through a separately authorized, auditable apply path;
- schedule refresh cadence by domain and project need rather than treating all scientific knowledge as equally volatile;
- enforce resource, provider, crawl-depth, token, and cost ceilings independently from user-facing experiment services;
- record which claim, source, retrieval pack, model, or distilled artifact was consumed by which envelope, candidate, calculation, experiment plan, decision, report, or release package.
- create canonical, rights-reviewed training exports from accepted measurements without merging incompatible chemistry families, methods, endpoints, or exposure conditions;
- maintain project-local, customer-authorized shared, chemistry-family, and global knowledge/model scopes separately;
- promote reusable campaign experience only after privacy, rights, applicability, leakage, and negative-transfer review;
- preserve explicit non-sharing decisions so project evidence cannot enter shared knowledge, model training, evolutionary seeds, operator statistics, or cross-project reports without authorization.

Measured lab evidence, retrieved literature, deterministic calculations, synthetic examples, and LLM-distilled material must remain separately typed. LLM output may propose summaries, hypotheses, normalized claims,

queries, or synthetic evaluation cases, but it is not empirical evidence and cannot raise scientific authority without independent source support and review.

Scientific Library

- A consolidated corpus of scientific literature, standards, patents, supplier documents, measured results, and reviewed internal knowledge to support formulation design and evolutionary experiments.
- Background discovery and freshness polling for materials relevant to active research projects, using project-scoped queries, source policies, crawl budgets, and deterministic stop conditions.
- Sparse and dense retrieval over stable approved knowledge, with candidate corpora available only to refinery evaluation and review surfaces.
- Content gained through governed distillation of capable external LLMs, stored as explicitly model-derived candidate material with provider/model identity, prompt/job identity, timestamp, cost, source context, and lineage.
- Provenance, rights, source class, content hashes, citations, update history, freshness, disagreement status, review state, audit receipts, supersession, and rollback boundaries for key items.
- Consumption lineage showing exactly what was consumed where, including retrieval query, selected evidence, candidate/run/experiment/report identifiers, and the version of every knowledge pack or model involved.
- Promotion and retirement controls so new or changed material cannot silently alter live generation, ranking, scientific claims, or release decisions.

Extensible Capability-Node Platform

The baseline specialist-module set is the initial coatings-oriented capability library, not a closed product boundary. Projects will introduce new material classes, representations, properties, instruments, simulations, manufacturing processes, evidence standards, and qualification paths. The platform must discover which capabilities a project needs, match those needs to eligible existing nodes, expose gaps, and support governed creation and reuse of new nodes.

“Node” is the product-level unit of composable capability. Existing modules may be packaged as nodes, but a node is not defined merely by a repository directory, executable, model, or prompt. It is a versioned, deployable, scientifically scoped capability with typed contracts, operational requirements, evidence posture, and lifecycle governance.

Node concepts

The platform must distinguish:

- **Node definition** — the stable capability identity and contract describing what question it can answer.
- **Node version** — an immutable implementation, schemas, dependencies, tests, evidence, authority, and compatibility declaration.
- **Node deployment** — a node version installed in a particular environment with runtime configuration, health, capacity, and credentials.
- **Node instance** — a node version selected and configured for a specific customer, project, experiment, or workflow graph.
- **Node execution** — one candidate-, sample-, experiment-, or task-bound invocation with exact inputs, outputs, provenance, and disposition.
- **Node result** — the typed evidence envelope emitted by an execution, including `PASS`, `FAIL`, `ABSTAIN`, `NOT_APPLICABLE`, or `UNKNOWN` as allowed.
- **DomainPack** — a reviewed set of representations, nodes, knowledge, data, methods, rules, benchmarks, and workflow templates for a material/application domain.

- **CapabilityPlan** — the reviewed node and workflow graph selected for one signed `ProductEnvelope` and decision objective.
- **CapabilityGap** — a typed statement that no eligible node or node composition can presently satisfy a required scientific, operational, evidence, or authority need.

Node definition contract

Every node definition/version must declare:

- stable node identity, display name, semantic version, owner, support contact, and lifecycle state;
- the narrow scientific, data, assurance, operational, or integration question it answers;
- accepted candidate/material/representation types and typed input/output schemas;
- domain, chemistry-family, formulation-space, process, substrate, method, endpoint, exposure, instrument, and jurisdiction applicability as relevant;
- explicit non-applicability and abstention conditions;
- method, equation, simulator, model, ruleset, external tool, instrument adapter, or composite implementation identity;
- required upstream nodes, datasets, knowledge packs, models, reference standards, instruments, providers, and environment capabilities;
- deterministic/non-deterministic posture, uncertainty/error behavior, limitations, failure modes, and required controls;
- evidence tier, scientific validation state, decision-use ceiling, and qualification/authority limitations;
- security classification, tenant compatibility, network behavior, data egress, secrets, licensing, intellectual-property, export-control, and third-party constraints;
- compute, memory, storage, accelerator, latency, throughput, cost, concurrency, and scaling profile;
- known-answer, negative-control, property/invariant, adversarial, privacy, security, performance, reproducibility, and domain-validation suites;
- artifact signatures, software bill of materials, build provenance, schema compatibility, migration posture, deprecation policy, and rollback boundary.

Node contracts and evidence envelopes must be machine-readable and UI-inspectable. Free-form documentation may explain a node but cannot replace its typed applicability, dependency, authority, and validation declarations.

Project-to-node matching

A signed `ProductEnvelope` and experiment/decision objective must compile into a `CapabilityNeedGraph` containing the properties, questions, decisions, methods, evidence, representations, constraints, and authority required by the project.

The capability planner must match needs against the node registry using:

- material and representation compatibility;
- property, constraint, method, endpoint, exposure, and decision applicability;
- available inputs, instruments, datasets, knowledge, and upstream results;
- required uncertainty, validation, evidence tier, and authority ceiling;
- customer/tenant authorization, confidentiality, rights, residency, and deployment restrictions;
- node operational health, version support, capacity, cost, schedule, and provider availability;
- schema, dependency, runtime, and domain-pack compatibility;
- independent-method, redundancy, control, or separation-of-duties requirements.

The resulting `CapabilityPlan` must show:

- each required capability and the node/version or reviewed composition selected to satisfy it;
- why the node is applicable and which exact inputs and dependencies it requires;

- alternatives considered and the reason each was rejected, deferred, or retained as an independent method;
- predicted cost, runtime, capacity, external dependency, evidence, uncertainty, and authority posture;
- gaps that block the workflow versus gaps that reduce coverage or confidence;
- required human, laboratory, customer, reviewer, or external-authority gates;
- the exact project lockfile used for execution.

Node matching may recommend but must not silently activate a node, relax applicability, substitute a cheaper/lower-authority method, or invent an answer where a capability is absent. An unsatisfied need produces a `CapabilityGap` and an explicit workflow consequence.

Capability-gap and node-build workflow

The system must support:

```
CapabilityGap
  -> NodeBuildRequest
  -> reviewed method and evidence plan
  -> scaffold/implementation
  -> local verification
  -> scientific and operational evaluation
  -> security/rights/licensing review
  -> shadow deployment
  -> limited-active deployment
  -> approved registry version
  -> eligible project lockfile
```

A `NodeBuildRequest` must contain:

- originating customer/project/envelope and the scientific or operational question;
- decisions the node may influence and requested authority;
- expected representations, inputs, outputs, units, evidence envelope, and failure/abstention behavior;
- intended material, method, process, endpoint, and exposure scope;
- candidate implementation methods and authoritative references;
- available and missing validation data, controls, benchmarks, and independent review;
- acceptance, refutation, uncertainty, reliability, security, privacy, performance, and cost criteria;
- urgency, budget, owner, support expectation, dependencies, and reuse potential;
- customer-private, customer-shareable, domain-pack, or global publication posture;
- IP ownership, licensing, data rights, export controls, confidentiality, and downstream-use restrictions.

Agents may draft node contracts, generate scaffolds and adapters, implement bounded code, assemble tests, and prepare evaluation packets. Agent-generated output remains a candidate version and cannot promote itself, expand its authority, or bypass scientific, security, licensing, privacy, operational, or customer review.

Node developer experience

The platform must provide a supported node SDK and development kit containing:

- templates for deterministic calculators, empirical models, simulators, rule engines, instrument adapters, data transformations, assurance checks, and composite nodes;
- canonical input/output, evidence, error, applicability, health, configuration, telemetry, and lifecycle schemas;
- local harnesses, representative synthetic fixtures, known-answer patterns, negative controls, fuzz/property tests, and authority-escalation tests;
- dependency and compatibility validation, reproducible builds, artifact signing, software-BOM generation, and registry packaging;
- isolated development and evaluation sandboxes that prohibit production/customer access by default;

- documentation generation, example UI projections, migration helpers, and backwards-compatibility checks;
- test doubles for upstream/downstream nodes and contract-testing against supported orchestrator versions;
- evaluation-packet and promotion-packet builders with complete provenance.

A node must be independently testable outside a complete campaign. Node development must not require editing the orchestrator core unless the public node contract itself changes.

Registry, composition, and dependency resolution

The node registry must support capability, applicability, schema, domain-pack, validation, authority, owner, lifecycle, deployment, cost, and health search. It must maintain signed immutable versions, aliases only where unambiguous, and explicit supersession/deprecation relationships.

Dependency resolution must detect:

- missing or cyclic dependencies;
- incompatible input/output schemas, units, representations, methods, or domain scopes;
- conflicting version or environment requirements;
- absent datasets, knowledge packs, models, instruments, providers, licenses, or credentials;
- duplicate calculations or evidence being mistaken for independent corroboration;
- authority chains that cannot satisfy the requested decision;
- nodes whose customer, residency, network, rights, or deployment posture conflicts with the project.

Composite nodes and workflow graphs must preserve the result, evidence, uncertainty, applicability, and authority of every constituent node. Composition cannot average away an abstention, promote weak evidence, hide disagreement, or raise the composite above the authority of its eligible inputs and reviewed policy.

Every campaign or experiment must bind an immutable capability lockfile naming exact node, schema, dependency, knowledge, dataset, model, rule, tool, and domain-pack versions. Re-resolution creates a new reviewed lockfile and never changes a historical run.

Node lifecycle and authority

Operational maturity and scientific authority are independent dimensions.

Operational lifecycle	Meaning
Draft	In development; not eligible for shared execution
Evaluated	Contract and required local tests pass
Shadow	Runs without affecting live decisions
Limited active	Eligible for explicitly scoped projects and decisions
Active	Supported for its declared operational scope
Deprecated	Supported temporarily with a migration target
Quarantined	Disabled because integrity, safety, security, scientific, or operational posture is unacceptable
Retired	No longer selectable for new lockfiles; retained for historical replay where possible

Scientific state must separately record unassessed, sanity-checked, benchmark-evaluated, domain-validated, decision-eligible, externally accepted where applicable, refuted, and superseded postures. A healthy deployment is not scientific validation, and scientific credibility does not imply current operational availability.

Promotion, demotion, quarantine, deprecation, migration, and retirement require durable decisions, affected-consumer analysis, notification, rollback/migration plans, and registry/audit updates. Emergency quarantine may occur before full review when continuing execution is unsafe, but it cannot delete historical evidence.

Domain packs and reuse

The initial coatings node set should be treated as the first `DomainPack`, combining candidate/evidence contracts, formulation arithmetic, ingredient knowledge, molecular/formulation gates, cure/viscosity/solvency/corrosion/process concepts, optimization controls, custody, sourcing/EHS/IP screening, replay, and reporting.

Future material domains may require different representations, simulators, tests, failure modes, process models, and manufacturing/qualification workflows. Batteries, membranes, ceramics, cement, composites, semiconductor materials, and biological formulations must not inherit coatings assumptions merely because a similarly named property exists.

Reuse must occur at the narrowest justified level: global infrastructure and vocabulary, cross-material methods, domain-pack capabilities, chemistry-family methods, customer-authorized nodes, and project-local adaptations. Promotion to a broader scope requires evidence at that broader scope and cannot be inferred from successful use in one project.

Capability-node acceptance posture

The node platform is not accepted until:

- every active node has a typed definition, immutable version, owner, applicability, dependencies, validation, authority, security/rights, resource profile, and lifecycle state;
- a signed project envelope can produce a reviewable capability-needs graph, matched plan, explicit gaps, and immutable execution lockfile;
- missing capability produces a typed gap rather than fabricated output or silent method substitution;
- a new node can be scaffolded, evaluated, packaged, shadowed, promoted, deployed, observed, rolled back, deprecated, and retired without changing the orchestrator core;
- operational health and scientific authority remain separately visible and enforceable;
- composite graphs preserve constituent evidence, disagreement, uncertainty, abstentions, provenance, and authority;
- customer-private nodes and data remain isolated, while broader reuse requires explicit rights and reviewed promotion;
- historical executions remain reproducible or explicitly bounded after node supersession, deprecation, quarantine, or retirement.

Enterprise Platform Requirements

Enterprise readiness requires explicit, measurable platform contracts in addition to scientific functionality. These requirements apply across every domain pack, node, customer, project, UI slice, agent, API, and deployment model.

Enterprise identity and access management

The product must support:

- enterprise federation through OIDC and SAML where required;
- multi-factor authentication and configurable conditional-access/session policy;
- SCIM or equivalent automated user/group provisioning and deprovisioning;

- role-based and attribute/policy-based authorization over tenant, customer, project, experiment, node, data class, action, environment, and authority;
- customer-admin delegated membership and role management within strict tenant boundaries;
- workload identities, service accounts, agent identities, credential rotation, expiration, and non-human access review;
- joiner, mover, leaver, suspended-user, emergency-access, and account-recovery workflows;
- periodic access certification, stale-access detection, separation-of-duties conflicts, and complete authorization-decision audit;
- API/session revocation, device/session visibility, and configurable authentication event notifications.

Authentication does not confer scientific, customer-data, operational, or approval authority. Authorization decisions must use the active tenant, purpose, object classification, delegation chain, environment, and action.

API and integration platform

The product must provide stable, documented, versioned APIs and event integrations for customers and internal services. Requirements include:

- explicit compatibility and deprecation policy for APIs, schemas, events, SDKs, and node contracts;
- idempotency, pagination, filtering, bulk operations, rate limits, quotas, correlation identities, and consistent error contracts;
- webhook/event subscriptions with signing, replay protection, delivery history, retries, reconciliation, and dead-letter handling;
- scoped service identities and OAuth/API credential management;
- supported SDKs, command-line automation, OpenAPI or equivalent machine-readable contracts, and contract-test suites;
- import/export and adapters for LIMS, ELN, SDMS, ERP, PLM, QMS, procurement/supplier systems, identity providers, object stores, messaging, and customer data platforms as needed;
- laboratory instrument and CRO adapters with format/version identity, validation, reconciliation, and raw-file preservation;
- integration health, dependency status, throughput, lag, failures, retries, schema drift, ownership, and support posture;
- customer-specific integrations isolated from shared defaults and promoted for reuse only through reviewed packaging.

Quantitative non-functional requirements

Each supported service tier and deployment profile must publish measurable objectives for:

- interactive API/UI availability and latency;
- background queue time, start time, throughput, maximum duration, and completion reliability;
- simultaneous customers, projects, experiments, campaigns, node executions, agents, users, and instrument/data imports;
- maximum and typical dataset, raw-file, artifact, request, result, and audit-export sizes;
- database, object-store, retrieval-index, audit-index, and UI projection freshness;
- backup frequency, backup verification, recovery point objective, recovery time objective, and disaster-recovery exercise frequency;
- durability and maximum acceptable loss for evidence, custody, audit, approval, and external-side-effect records;
- retention, export, legal-hold, and verified deletion completion times;
- incident detection, acknowledgement, mitigation, customer notification, and post-incident reporting;
- agent queue, response, intervention, escalation, recovery, cost, and correctness by workflow criticality;
- planned degraded behavior when a model, provider, dataset, node, laboratory, integration, or region is unavailable.

Targets may differ by workflow and tier. Interactive formulation preview, long-running simulation, laboratory return, agent research, and qualification packaging must not be assigned one misleading latency or availability target.

Supported deployment models

The product strategy must explicitly designate which profiles are supported, planned, or out of scope:

- multi-tenant SaaS;
- single-tenant hosted SaaS;
- customer private-cloud/VPC deployment;
- customer-managed on-premises deployment;
- disconnected or air-gapped deployment;
- hybrid execution near instruments, laboratories, controlled data, or high-performance computing.

Each supported profile must define ownership and support boundaries, installation/upgrade/rollback, observability, backup/recovery, identity integration, key management, egress, model/provider availability, data residency, node compatibility, offline licensing, update delivery, and incident handling. A project capability plan must exclude nodes incompatible with its deployment profile.

Security assurance and software supply chain

The product must include:

- documented security ownership, secure-development lifecycle, threat modeling, security architecture review, and risk acceptance;
- dependency, container, infrastructure, secret, static, dynamic, and supply-chain scanning appropriate to each component;
- signed builds/releases/node packages, build provenance, software bills of materials, dependency/license inventory, and verification at deployment;
- vulnerability intake, severity classification, remediation objectives, compensating controls, disclosure, and customer communication;
- independent penetration/adversarial testing and tracked remediation;
- environment and network segmentation, egress policy, runtime hardening, least privilege, encryption in transit/at rest, and managed-key rotation;
- customer-managed or dedicated encryption keys where required by the supported tier;
- security incident response, forensic preservation, containment, credential rotation, recovery, notification, and lessons-learned workflows;
- third-party/provider risk inventory, data/process/subprocessor scope, availability/security posture, substitution plan, and contract dependencies;
- export-control, sanctions, IP, licensing, privacy, and jurisdictional checks appropriate to customer and material domain;
- compliance evidence packages and control mapping appropriate to the markets pursued, without representing internal controls as external certification.

Business continuity and service management

The product must support:

- service catalog, ownership, dependency map, operational tier, support hours, escalation, and service-level objectives;
- public/customer-appropriate status and incident communication separated from privileged operational detail;

- capacity planning, load and failure testing, regional/provider dependency analysis, and documented scaling limits;
- rehearsed backup restoration, regional/service failover, dependency outage, credential-loss, and key-person/runbook scenarios;
- incident, problem, change, release, maintenance, and post-incident workflows with customer-impact tracking;
- support cases linked to customers, incidents, tasks, experiments, agents, deployments, and delivered artifacts;
- business-continuity plans with named owners, recovery priorities, manual fallbacks, communication paths, and exercise evidence;
- explicit RPO/RTO and data-integrity posture for each authoritative store and supported deployment profile.

Commercial entitlements, metering, and cost governance

The customer and platform model must support:

- contracts, plans, effective dates, feature/node/model/data entitlements, seats, roles, environments, support tiers, and service levels;
- customer/project budgets and quotas for compute, storage, data transfer, tokens, providers, agents, node executions, experiments, and retained artifacts;
- accurate tenant/project/task/node/provider usage metering and allocation without exposing another tenant's usage;
- forecast, threshold alert, approval, hard-stop, overage, internal chargeback, invoice/export, and dispute/reconciliation workflows as required;
- trial, evaluation, sandbox, suspended, grace-period, renewal, termination, and offboarding states;
- node- and provider-specific licensing restrictions, offline entitlements, concurrency, geography, and expiration;
- safe behavior when an entitlement expires or a quota/budget is exhausted, preserving evidence and avoiding partial external side effects;
- auditability of entitlement changes and identification of affected scheduled/running work.

Commercial status cannot retroactively alter the historical evidence or authority of completed work. Restricted access or offboarding must follow contractual retention, export, deletion, and legal-hold obligations.

Configuration, customization, and change management

- Tenant/project configuration must be layered, typed, validated, versioned, reviewable, exportable, and distinguishable from product defaults.
- Customer terminology, workflows, fields, policies, reports, notifications, units, domain packs, nodes, and integrations may be configured only through supported extension points.
- Configuration changes require preview, affected-consumer analysis, authority/approval where needed, audit, activation time, rollback, and historical binding.
- Feature flags must have owner, scope, purpose, audience, environment, default, expiration/review date, dependencies, telemetry, and rollback behavior.
- Schema and data migrations require compatibility plans, rehearsal, validation, observability, rollback/forward-repair posture, and customer-impact communication.
- Unsupported customer forks must not become an invisible operational dependency; exceptions require ownership, support posture, upgrade plan, and retirement trigger.

Accessibility, localization, and adoption

The product must define and verify:

- accessibility targets, keyboard navigation, screen-reader semantics, focus/error behavior, contrast, reduced-motion support, and accessible alternatives to visualizations;
- supported browsers, screen sizes, assistive technologies, and performance on representative enterprise devices/networks;
- canonical unit storage with user/project-selectable display units, lossless conversion history, locale-aware numbers/dates, and unambiguous scientific notation;
- time-zone behavior using absolute timestamps and visible source/display zones for custody, audit, scheduling, and external coordination;
- localization/internationalization posture for UI, reports, notifications, controlled terminology, and customer content;
- role-based onboarding, contextual help, guided workflows, sample/sandbox projects, administrator/operator runbooks, API/node developer documentation, and release notes;
- training, competency, acknowledgment, and re-training workflows where process or regulated use requires them;
- privacy-aware product analytics and user-feedback mechanisms with tenant controls and no silent reuse as scientific evidence.

Enterprise platform acceptance posture

Enterprise readiness is not accepted until:

- federated identity, automated provisioning, least-privilege authorization, access review, and separation of duties work across UI, API, agents, nodes, and support access;
- every supported deployment profile has owned installation, upgrade, rollback, security, observability, backup, recovery, and support procedures;
- quantitative capacity, latency, availability, durability, RPO/RTO, and degraded-mode targets are tested and monitored;
- public APIs/events have compatibility policy, authentication, rate/idempotency behavior, contract tests, and operational reconciliation;
- releases and node packages have verified provenance, signatures, software bills of materials, vulnerability posture, and rollback paths;
- commercial entitlements, quotas, budgets, usage, and expiration safely govern work without corrupting evidence or causing ambiguous side effects;
- accessibility, units, time zones, localization posture, onboarding, documentation, and supported-client behavior are tested with intended users;
- enterprise offboarding can export, retain, legally hold, revoke, and delete the correct scoped material with verifiable completion.

User Experience and Role Workspaces

The product UI must be organized into role-aware and workflow-aware slices. A user may hold more than one role, but permissions, default views, terminology, and available actions must reflect the active role and tenant context. Role switching must not weaken authorization or conceal which identity performed an action.

Shared UI requirements across all slices:

- persistent customer, project, experiment, campaign, candidate, sample, evidence, and agent context in navigation;
- explicit environment, tenant, authority, data-classification, and read/write posture;
- consistent status, blocker, owner, next-action, freshness, and last-event summaries;
- source-to-decision traceability and drill-down from summary claims to exact evidence, rules, calculations, models, prompts, agent actions, and approvals;
- accessible tables and visualizations with exportable machine-readable equivalents;

- notifications and task inboxes scoped by role, severity, customer, project, and required response;
- no hidden agent or automation state: pending, running, waiting, retrying, failed, blocked, cancelled, superseded, and completed states must be visible;
- saved views and deep links that preserve tenant and authorization boundaries;
- immutable audit history for consequential changes, reviews, approvals, overrides, and exports.

Operator and DevOps Slice

The Operator is also the DevOps role and owns platform reliability, deployment, runtime configuration, capacity, incident response, and supported recovery. This role does not automatically receive scientific, customer-data, or qualification authority.

The slice must provide:

- fleet, environment, service, worker, queue, scheduler, storage, database, model-serving, retrieval, and agent-runtime health;
- cross-customer operational status using metadata-minimized views that do not expose customer scientific content without separate authorization;
- deployment version, configuration version, schema/migration status, dependency health, feature flags, and rollback controls;
- crashes, errors, retries, dead letters, stalled jobs, unavailable dependencies, latency, throughput, compute, memory, storage, network, token, provider, and cost telemetry;
- workload admission, priorities, quotas, concurrency, resource reservations, maintenance windows, and safe cancellation/draining;
- backup, restore, disaster-recovery, integrity-check, retention, deletion, and legal-hold operations with rehearsed recovery evidence;
- secrets, keys, service identities, certificates, provider credentials, rotation status, and least-privilege access without rendering secret values;
- incident timelines linking alerts, affected services/jobs/agents, operator actions, mitigations, follow-up work, and customer-impact posture;
- controlled diagnostic access, support bundles, redaction previews, and break-glass workflows with reason, duration, approval, and complete auditing;
- agent pause, resume, drain, quarantine, retry, reroute, and terminate controls that preserve evidence and distinguish operational intervention from scientific approval;
- release preflight, canary/shadow status, regression results, rollback boundary, and post-deployment verification.

External Customer Slice

The external customer experience must expose program progress and decisions without revealing other tenants, internal-only deliberation, protected model details, privileged security information, or unsupported scientific authority.

The slice must provide:

- customer organization, program, project, milestone, experiment, deliverable, decision, and action-item overview;
- secure source upload and structured intake for decks, technical envelopes, specifications, emails, notes, samples, constraints, and starter prompts;
- extracted-requirement review with confirm, correct, reject, annotate, needs-source, needs-customer-answer, and defer actions;
- side-by-side source text and proposed structured fields including unit, tolerance, method, priority, provenance, confidence, assumption, and authority;

- ambiguity register, clarification requests, decisions required, response deadlines, and downstream blocking effects;
- approval surfaces for customer-owned requirements, intended use, constraints, information-sharing posture, milestones, and deliverables;
- experiment status and approved result summaries with clear distinctions among planned, modeled, measured, internally screened, evaluation-ready, and externally qualified states;
- candidate comparisons and reports phrased within evidence and authority limits, with limitations and unresolved risks visible;
- secure messages, document exchange, comment resolution, approval history, and notification preferences;
- configurable sharing and consent controls governing whether project evidence may enter reusable knowledge, training, model evaluation, evolutionary seeds, or cross-project statistics;
- export and delivery history showing artifact versions, recipients, watermark/classification, approvals, expiration, and revocation where supported.

Internal Data Scientist Slice

The internal data scientist owns data suitability, feature and dataset construction, statistical modeling, evaluation, monitoring, and promotion recommendations. This role does not independently grant customer, production, safety, or qualification authority.

The slice must provide:

- dataset catalog, source rights, schema, lineage, splits, transformations, quality findings, missingness, bias, drift, applicability, and consumption history;
- project-local and reusable knowledge/training scope with explicit customer authorization and exclusion controls;
- experiment and measurement exploration across compatible projects without flattening chemistry family, method, endpoint, substrate, process, exposure, or authority differences;
- notebook or job launch through governed, reproducible environments with pinned code, dependencies, data snapshots, seeds, and resource budgets;
- feature definitions, training-set assembly, leakage checks, negative controls, uncertainty analysis, out-of-domain behavior, and reproducible evaluation;
- model, adapter, embedding, retrieval-pack, surrogate, and policy registry views with versions, training lineage, metrics, limitations, approvals, consumers, and rollback state;
- candidate-versus-stable comparison, shadow/limited-active evaluation, regression suites, promotion packet authoring, and retirement workflows;
- sparse/dense retrieval inspection including query, retrieved material, reranking, evidence status, rights, freshness, disagreement, and downstream consumption;
- neuro-symbolic inference inspection showing probabilistic hypotheses, governing rules, supporting/conflicting evidence, alternatives, unknowns, and abstentions;
- evolutionary analysis showing populations, diversity, operators, fitness components, uncertainty, Pareto fronts, rejects, duplicates, selection decisions, and prior-project influence;
- monitoring for drift, calibration, applicability failures, data-quality regressions, unexpected abstention, model/provider changes, and customer-scope violations.

Agent Management Slice

The agent-management UI is the control plane for agent identities, capabilities, work, supervision, and evidence. It must not reduce agents to an uninspectable chat transcript.

The slice must provide:

- agent registry with role, purpose, owner, model/provider, version, prompt/policy pack, tools, data scopes, tenant scopes, authority ceiling, budget, and lifecycle status;

- topology showing orchestrators, specialists, reviewers, support agents, dependencies, delegation chains, current assignments, and communication paths;
- live and historical task views containing objective, plan, inputs, outputs, checkpoints, tool calls, costs, elapsed time, retries, blockers, approvals, and terminal disposition;
- work queues, schedules, priorities, service-level targets, concurrency, leases, heartbeats, cancellation, pause/resume, and deterministic retry controls;
- capability and tool-permission editor using least privilege, environment separation, credential references, network policy, write boundaries, and expiration;
- approval inbox for proposed destructive, external, high-cost, cross-tenant, scientific-authority, knowledge-promotion, model-promotion, and release-affecting actions;
- handoff and escalation surfaces showing what the agent knows, what it attempted, why it stopped, what authority/input is missing, and the exact safe continuation point;
- transcript, structured event, artifact, prompt, context-source, retrieval, tool, and decision lineage with redaction and retention controls;
- agent evaluations, known-answer tests, adversarial tests, reliability, cost, latency, intervention rate, policy violations, and regression history;
- canary, shadow, limited-active, active, quarantined, deprecated, and retired deployment states with promotion and rollback artifacts;
- emergency global and tenant-scoped stop controls that interrupt new work safely without deleting evidence or leaving ambiguous external side effects.

Customer Slice

The customer slice is the internal system of record for customer ownership, isolation, commercial/program context, permissions, and the portfolio of work performed for that customer.

The slice must provide:

- `customer -> program/project -> experiment/campaign -> run` hierarchy with collision-safe identities;
- customer contacts, roles, contracts, data rights, confidentiality, jurisdictions, intended uses, retention, deletion, sharing consent, and communication preferences;
- tenant-scoped members, groups, service identities, role assignments, access reviews, and break-glass history;
- portfolio status across all customer experiments, milestones, deliverables, blockers, risks, decisions, budgets, compute consumption, and agent activity;
- customer-specific ingredient, supplier, plant, process, standards, model, knowledge, and policy applicability without leaking these records into global defaults;
- data residency, encryption, storage, backup, export, legal-hold, and deletion posture;
- separation of customer-visible artifacts, internal working material, privileged reviews, and restricted evidence;
- reusable-knowledge and cross-project-learning consent ledger with precise scope, effective dates, revocation, and downstream consumption impact;
- customer timeline containing intake, approvals, experiments, reports, incidents, changes, exports, and handoffs.

Experiment Slice

The experiment slice is the full scientific lifecycle view for characterization, formulation, candidate, laboratory, modeling, and evolutionary work.

The slice must provide:

- charter, hypothesis, decision objective, `ProductEnvelope`, scope, authority, owners, milestones, budget, and blocking ambiguities;

- lifecycle visualization spanning intake, characterization, baseline reconstruction, candidate generation, preflight, protocol, execution, return intake, review, analysis, selection, reporting, and closure;
- immutable formulation/candidate versions, lineage, BOM, process, sourcing, cost, evidence tier, limitations, and comparisons;
- samples, aliquots, fractions, preparation, custody, methods, instruments, calibrations, controls, raw files, observations, deviations, and dispositions;
- protocol and test-matrix authoring, CRO/lab issue and acknowledgement, scheduling, execution status, return reconciliation, and amendments;
- live deterministic properties, constraint margins, specialist outcomes, applicability, uncertainty, abstentions, and source/calculation drill-down;
- characterization hypotheses, mass balance, unidentified fraction, analytical fingerprints, contradictions, baseline reconstructions, and acceptance gates;
- genetic/evolutionary visualization including seeds, generations, parentage, mutations/recombinations, diversity, fitness evidence, Pareto views, rejects, duplicates, and reviewed selections;
- results visualization with replicate and control context, uncertainty, method comparability, failure analysis, and separation of predicted versus measured values;
- agent activity associated with the experiment, including proposed actions, work in progress, completed artifacts, required approvals, failures, and escalations;
- structured feedback and sign-off for extracted fields, assumptions, soft-objective weights, allowed variables/operators, campaign launch, evidence disposition, next batch, and final deliverables;
- complete feedback history showing who or which agent changed what, when, why, under what authority, and which downstream artifact consumed the decision.

Cross-Domain UI and Functional Auditability

Auditability is a product behavior and domain contract, not a reporting feature added after execution. Every consequential workflow must be inspectable through the UI and reconstructable from durable machine-readable records across customer, project, experiment, sample, formulation, candidate, campaign, evidence, knowledge, model, agent, infrastructure, support, and release domains.

Common audit event contract

Every consequential state transition, decision, calculation, agent action, automated action, approval, external side effect, import, export, and administrative intervention must emit an immutable audit event containing, as applicable:

- globally unique event identity, event type/schema/version, timestamp, sequence, and correlation/causation identities;
- tenant, customer, project, experiment, campaign, run, candidate, sample, evidence, model, agent, task, and environment scope;
- human, agent, service, or external-system actor identity and the authenticated/delegated identity chain;
- action requested, action performed, target object and version, prior state/hash, resulting state/hash, and changed fields;
- reason, source request, governing policy/rule, authority ceiling, permission decision, and approval or denial references;
- input, configuration, prompt/policy, knowledge, dataset, model, code, dependency, tool, method, and instrument versions or content hashes;
- output artifacts, external side effects, recipients/endpoints, costs, errors, retries, warnings, abstentions, limitations, and terminal disposition;
- confidentiality, rights, retention, legal-hold, redaction, and customer-sharing classifications;
- integrity protection sufficient to detect deletion, alteration, reordering, substitution, or cross-tenant misbinding.

An event may reference protected payloads rather than duplicate them, but the reference must be immutable, access-controlled, retention-aware, and sufficient to establish exactly which bytes or domain version were used.

Domain-object history and state reconstruction

Every first-class domain object must expose:

- creation identity, creator, source, timestamp, owning tenant/project, schema version, and initial authority posture;
- current state plus ordered history of transitions, reviewers, approvals, rejections, overrides, supersessions, and retirements;
- immutable referenced versions for any object consumed by an experiment, calculation, campaign, report, promotion, or release decision;
- parent-child, input-output, derived-from, supersedes, consumed-by, and affected-by relationships;
- field-level or semantically grouped diffs between versions, including units and normalization/conversion history;
- unresolved ambiguities, contradictions, deviations, missing evidence, abstentions, and policy exceptions;
- the ability to reconstruct state at a selected event or time without silently substituting current knowledge, models, policies, or source content.

Mutable operational projections may be rebuilt for performance, but they cannot replace the authoritative event, version, and artifact history.

Decision and calculation auditability

Every scientific, business, policy, model, agent, or operational decision shown by the product must answer:

1. What was decided or calculated?
2. Who or what made and approved it?
3. Which exact inputs, versions, evidence, rules, equations, models, prompts, tools, and assumptions were used?
4. Which alternatives, contradictions, uncertainty, missing information, abstentions, or failed gates were present?
5. Why did the result pass, fail, rank, abstain, change, or require escalation?
6. What downstream objects, actions, reports, customers, experiments, models, or releases consumed it?
7. Can the deterministic portion be recomputed, and can the non-deterministic portion be meaningfully replayed or bounded?
8. What authority and applicability limits attach to the result?

Deterministic calculations must retain equation/calculator version, declared inputs and units, conversions, intermediate values needed for verification, rounding policy, result, and test/validation identity. Stored computed values must identify whether they remain reproducible from retained inputs; recomputation must never overwrite the historical result consumed by a past decision.

Model and LLM-assisted outputs must retain provider/model/version, prompt or structured request identity, system/policy pack, context and retrieval identities, parameters where available, output, parsing/validation result, uncertainty or confidence posture, review state, and downstream use. Provider opacity must be recorded as a limitation rather than represented as deterministic reproducibility.

Evidence and scientific-claim auditability

- Every claim must distinguish customer-stated, measured, deterministic, modeled, retrieved, supplier-stated, assumed, inferred, model-generated, and externally authorized evidence types.

- Claims must bind supporting and conflicting evidence, exact method and scope, confidence/uncertainty, freshness, rights, applicability, reviewer disposition, and authority ceiling.
- Raw evidence, normalized observations, derived features, calculations, interpretations, and report language remain separately addressable.
- Sample, aliquot, fraction, preparation, custody, instrument, calibration, method, raw-file, observation, deviation, and reviewer lineage must be traversable in both directions.
- A user must be able to identify every decision or artifact affected when evidence is corrected, rejected, superseded, retired, rights-blocked, or found stale.
- Qualification, safety, procurement, release, and performance representations must expose the external authority or explicitly show that no such authority exists.

UI audit experience

Every role-appropriate detail view must include an audit entry point. The UI must provide:

- a chronological timeline and a dependency/lineage graph, with filters for actor, event, object, action, status, authority, evidence type, date, and correlation identity;
- current-versus-prior and arbitrary-version comparison with added, removed, changed, unchanged, and authority-impacting fields clearly identified;
- source-to-field, evidence-to-claim, input-to-calculation, candidate-to-result, decision-to-consumer, and agent-task-to-side-effect trace views;
- replay or recompute controls where authorized, with a preview of pinned inputs, expected side effects, cost, environment, and whether the operation creates a new result rather than altering history;
- visible human, agent, service, imported, calculated, and externally authorized authorship distinctions;
- clear indicators for stale, superseded, quarantined, rejected, unreviewed, assumed, out-of-domain, rights-limited, and authority-limited material;
- approval and override history including eligible approvers, separation-of-duties checks, comments, expiration, revocation, and downstream impact;
- tamper/integrity status and an explicit indication when audit completeness cannot be established;
- role-appropriate redaction that shows why content is unavailable without leaking the protected value;
- export of the filtered human-readable view and the underlying machine-readable event/lineage package under the same authorization controls.

Summary dashboards may aggregate audit state, but they must link to the underlying objects and must not flatten uncertainty, dissent, abstention, or authority limits into a misleading green status.

Search, reporting, and audit packages

Authorized users must be able to search audit records by object identity, actor, customer/project, experiment/campaign/run, sample, candidate, claim, source, model, agent, task, approval, artifact hash, external side effect, and correlation identity.

The system must create scoped audit packages for scientific review, customer delivery, incident investigation, model/knowledge promotion, security review, regulatory/quality review, and release readiness. Each package must contain a manifest, scope, generation identity/time, object and event inventory, content hashes, lineage, applicable redactions, omitted-content reasons, integrity verification instructions, and authority limitations.

Audit exports are immutable snapshots. Later corrections or supersessions produce new linked exports and must not silently rewrite a previously delivered package.

Security, privacy, and tenant boundaries

- Audit access is least-privilege and purpose-scoped; the ability to operate a system does not automatically grant access to customer scientific content.

- Audit records must preserve tenant isolation and prevent identifiers, search results, counts, timing, errors, or agent traces from leaking the existence or content of another customer's work.
- Sensitive payloads, secrets, credentials, privileged security detail, personal data, and restricted customer content must be separated from commonly searchable event metadata and protected through field/object-level controls.
- Authorized redaction or deletion must preserve a verifiable tombstone, legal basis, scope, actor, approval, time, and affected downstream references without retaining prohibited content.
- Break-glass audit access requires a reason, bounded scope and duration, notification/escalation policy, and an independently reviewable record.
- Audit administrators cannot alter scientific evidence, approvals, or operational history; maintenance and retention actions are themselves audited.

Reliability and failure posture

Consequential work must fail closed or enter an explicit non-final state when its mandatory audit event, artifact binding, actor identity, authorization decision, or integrity record cannot be durably written. The system must not report success while silently dropping audit data.

Event delivery may be asynchronous only when the originating transaction durably records an outbox or equivalent recoverable commitment. Duplicate delivery must be idempotently reconciled; missing sequences, clock anomalies, signature failures, hash mismatches, schema failures, and cross-scope bindings must alert and quarantine affected outputs as appropriate.

Audit storage, indexes, projections, and exports require backup, restore, retention, integrity verification, capacity monitoring, and disaster-recovery tests. Recovery must prove that event ordering, hashes, relationships, redactions, and tenant boundaries remain intact.

Auditability acceptance posture

Cross-domain auditability is not accepted until:

- every consequential workflow emits the common audit contract and links it to its domain objects;
- a reviewer can traverse from a displayed result to exact inputs, evidence, rules/models, actors/agents, approvals, changes, and downstream consumers;
- arbitrary-version comparison and point-in-time reconstruction work without substituting newer inputs;
- deterministic results can be independently recomputed from retained artifacts, while non-deterministic limitations are explicit;
- evidence corrections, rights changes, model/knowledge retirement, and revoked approvals produce a reliable affected-consumer analysis;
- unauthorized users cannot infer another tenant's activity through audit search, UI, exports, errors, or operational telemetry;
- dropped, reordered, altered, duplicated, or misbound events are detected and handled without presenting an unauditable action as successful;
- audit packages are integrity-verifiable, access-controlled, reproducible as snapshots, and explicit about omissions and authority limits;
- UI usability testing demonstrates that intended reviewers can answer the eight decision-audit questions without direct database or log access.

Fully Agent-Orchestrated and Supported Environment

The product must support a fully agent-orchestrated operating model in which authorized agents can plan, coordinate, execute, monitor, diagnose, document, and support routine work across the product lifecycle. “Fully agent-orchestrated” means the system can carry an approved objective through its permitted workflow without

requiring humans to manually shuttle files, copy context, poll jobs, or reconstruct state. It does not mean unlimited autonomy or removal of accountable human decision-makers.

Agent roles and responsibilities

The environment should support specialized, composable agents such as:

- intake and requirement-extraction agents;
- characterization-planning and analytical-reconciliation agents;
- formulation, constraint-compilation, and candidate-generation agents;
- experiment-design, protocol, CRO/lab-coordination, and return-ingestion agents;
- data-curation, knowledge-refinery, retrieval, modeling, and evaluation agents;
- evolutionary campaign, monitoring, comparison, and reporting agents;
- platform operations, incident triage, cost/capacity, security, and customer-support agents;
- independent evaluator, policy-checking, evidence-review, and red-team agents.

Agent roles must remain separable. The system must not allow the agent proposing a consequential scientific or operational change to impersonate the required independent reviewer or approval authority.

Orchestration model

Every agent task must be represented by a durable work object containing objective, tenant/project/experiment scope, requester, assigned agent and version, inputs, constraints, authority ceiling, tools, data access, budget, deadline, plan/checkpoints, outputs, artifacts, dependencies, approvals, retries, and terminal disposition.

The orchestrator must:

- decompose approved objectives into dependency-aware work while preserving the original scope and authority;
- select agents based on declared capability, health, validation status, tenant eligibility, data clearance, cost, and workload;
- support sequential, parallel, conditional, review, voting, adversarial, and escalation patterns through explicit workflow definitions;
- pass typed references and immutable artifacts rather than relying on lossy transcript summaries as the system of record;
- persist state across restarts and support leases, heartbeats, idempotency, deduplication, resumability, backoff, retry limits, compensation, cancellation, and dead-letter handling;
- detect contradictory results, missing prerequisites, stale inputs, tool failures, loops, budget exhaustion, and authority gaps and route them to the correct reviewer or support queue;
- distinguish `WAITING_FOR_DEPENDENCY`, `WAITING_FOR_APPROVAL`, `WAITING_FOR_CUSTOMER`, `RETRYABLE_FAILURE`, `BLOCKED`, `CANCELLED`, `SUPERSEDED`, and `COMPLETE`;
- never broaden scope, tenant access, external side effects, spending authority, scientific authority, or data-sharing permission merely to complete a task.

Human control and approval

Humans remain accountable for customer commitments, signed envelopes, destructive or externally visible actions, material spend, customer-data sharing, knowledge/model promotion, scientific evidence disposition, campaign activation where required, release/qualification representations, and exceptions to policy.

The environment must provide:

- risk- and policy-based approval routing with named eligible approvers and separation of duties;

- previewable proposed action, affected objects, expected side effects, cost, evidence, uncertainty, rollback/compensation plan, and expiration before approval;
- approve, reject, modify, delegate, request-more-evidence, pause, cancel, and revoke actions;
- bounded standing approvals for routine low-risk work, with explicit scope, limits, duration, and revocation;
- interruption and emergency-stop behavior that leaves durable, reconcilable state;
- no approval laundering through agent consensus, repeated retries, prompt wording, or decomposition into smaller actions.

Tools, credentials, and execution safety

- Agents receive short-lived, least-privilege identities and tool grants scoped to tenant, project, environment, operation, resource, and time.
- Secrets remain in managed secret stores; prompts, transcripts, logs, reports, and artifacts contain references or redacted metadata rather than secret values.
- Development, test, shadow, limited-active, and production environments remain isolated with separate data and credentials.
- File, database, network, provider, laboratory, messaging, and deployment tools declare read/write/external/destructive effects and support policy interception.
- Potentially destructive or externally visible actions require target resolution, preflight, authorization, idempotency or compensation where possible, and post-action verification.
- Agent-generated code, prompts, models, rules, workflows, and configuration pass applicable tests, security checks, policy checks, and review before activation.
- Untrusted documents, retrieved content, customer material, and tool output are treated as data rather than executable instruction and must not override system policy or task authority.

Observability, provenance, and support

Every agent execution must emit structured, queryable events covering task state, agent/model/prompt/policy version, input and context identities, retrievals, tool requests/results, decisions, artifacts, token/compute/provider cost, latency, errors, retries, approvals, external side effects, and final disposition.

The environment must support:

- live operational views and alerts for stalled, looping, failing, expensive, policy-denied, or anomalous agents;
- end-to-end trace from a customer request or system event through agent work to every resulting scientific, operational, or customer-facing artifact;
- reproducible replay where deterministic inputs permit it and an explicit non-reproducibility record where providers or environments prevent exact replay;
- automated incident triage, evidence-preserving diagnostic collection, runbook selection, proposed remediation, and escalation to the Operator/DevOps role;
- customer-support agents that can summarize authorized status and known issues but cannot expose other tenants, internal secrets, privileged reasoning, or unsupported claims;
- service-level objectives for queue time, completion time, intervention rate, recovery, support response, cost, and correctness by workflow criticality;
- retention, redaction, export, deletion, legal-hold, and audit behavior consistent with customer and regulatory obligations.

Agent quality and lifecycle

Agents, prompt/policy packs, tools, and workflows are versioned deployable components. Each requires an owner, intended scope, authority ceiling, evaluation suite, known limitations, monitoring plan, and rollback

boundary.

Activation must progress through development, evaluation, shadow, limited-active, and active stages as risk requires. Promotion depends on known-answer, negative-control, adversarial, privacy, security, authority, reliability, cost, and domain-specific evaluations. Material changes trigger re-evaluation. Regressing or unsafe agents can be automatically quarantined without deleting their work history.

Agent evaluation must measure outcome correctness and evidence quality, not merely task completion or persuasive output. Required measures include unsupported-claim rate, correct abstention, escalation quality, tool/authority violations, data leakage, reproducibility, intervention rate, recovery behavior, latency, and cost.

Agent-orchestration acceptance posture

The environment is not fully agent-orchestrated until:

- an approved customer or internal objective can proceed through its authorized workflow without manual context/file shuttling;
- all work, dependencies, agents, tools, approvals, artifacts, costs, and external side effects are visible from one durable execution graph;
- restart, retry, cancellation, dependency failure, agent failure, provider failure, and human-wait states recover without ambiguous duplication or lost custody;
- every agent action is tenant-scoped, least-privilege, policy-checked, attributable, and auditable;
- consequential actions stop at the correct human or external-authority gate;
- operators can diagnose, pause, reroute, resume, or safely terminate work without corrupting experiment or evidence state;
- customers and internal users receive role-appropriate status and support without exposure of restricted data or unsupported authority;
- agent, prompt, tool, workflow, model, and knowledge changes have evaluated promotion and tested rollback paths.

Non-Goals

- Do not claim a candidate is qualified, safe, procurable, NAVSEA-approved, QPL-listed, or legally clear unless external authority exists.
- Do not silently use inferred/defaulted requirements as customer-approved constraints.
- Do not store computed properties as authoritative source data when they can be re-derived from formulation composition.
- Do not use unresolved-license data in ranking or promotion evidence.
- Do not optimize candidates outside plant, feedstock, cost, safety, or qualification constraints and then filter late.
- Do not enable chemistry-changing mutation or scientific selection until the relevant operators and fitness modules have authority.
- Do not let LLM-generated prose bypass `ProductEnvelope`, `EvolutionCampaign`, `FitnessComponent`, `SelectionDecision`, or evidence-intake contracts.
- Do not promote raw web text, search results, supplier claims, LLM output, synthetic examples, or unreviewed summaries directly into trusted retrieval, model-training, ranking, or release evidence.
- Do not allow the knowledge refinery to mutate live trusted stores, models, prompts, policies, or experiment-selection behavior without an explicit reviewed promotion artifact and rollback boundary.
- Do not present a normalized or model-distilled claim as measured fact; evidence type, scope, freshness, confidence, and limitations must remain visible downstream.
- Do not treat the baseline coatings module/node set as complete or universally applicable to other material domains.

- Do not allow a capability matcher, orchestrator, or agent to invent a missing node result, silently substitute a lower-authority method, or relax project requirements to make a plan executable.
- Do not allow a new or modified node to self-certify, self-promote, broaden its applicability, or inherit authority merely because it passes operational tests or worked in another project.
- Do not let node composition, ensemble logic, or graph aggregation hide disagreement, convert abstentions into scores, duplicate evidence as independent support, or exceed constituent authority.
- Do not make customer-specific configuration, nodes, evidence, models, or campaign experience globally reusable without explicit rights and reviewed promotion.

Acceptance Posture

Initial acceptance is a usable MVP, not scientific proof:

- Scientists can manage ingredients, formulations, experiments, and measurements.
- The physics engine recomputes deterministic coating properties from composition.
- The suggestion engine proposes next formulations with constraints respected.
- Formulation versions are immutable once referenced by experiments.
- Every candidate or suggestion carries provenance, limitations, and authority status.
- Architecture keeps physics/ML pure and swappable.
- Reverse-engineering campaigns can trace each sample, aliquot, fraction, preparation, method, instrument, calibration, raw file, normalized observation, and review decision.
- Incumbent characterization reports composition ranges, confidence, alternatives, contradictions, mass balance, unidentified fractions, and method limitations without claiming a uniquely recovered recipe.
- A versioned baseline reconstruction must pass reviewed fingerprint, processing, cure, and relevant performance gates before it becomes an optimization seed.
- Naval candidate packages distinguish internal screening, prequalification readiness, authorized evaluation, and external qualification; the framework cannot assign NAVSEA/QPL or equivalent external authority.
- The naval paint charter plus technical envelope can be compiled into a reviewed `ProductEnvelope` before any evolutionary run starts.
- Users can provide structured feedback on extracted envelope fields, ambiguity items, soft-objective weights, allowed variables, and operator permissions before campaign launch.
- Users can inspect the compiled symbolic search contract separately from evolutionary campaign state, and every constraint, objective, operator, fitness component, and selection decision retains typed authority and lineage.
- Neuro-symbolic explanations identify the governing rules and evidence, while evolutionary replay reproduces the population, operators, fitness inputs, rejects, duplicates, and selections from pinned inputs and seeds.
- Curators can inspect raw sources, normalized records, candidate claims, conflicts, freshness, evaluations, and promotion history without exposing candidate knowledge to live experiment workflows.
- Only approved evidence-backed claims and knowledge-pack versions can influence live candidate generation or experiment selection, and every use is traceable to its exact source and refinery lineage.
- A promoted knowledge change can be rolled back to a prior stable boundary and rebuilt deterministically from retained upstream artifacts.
- Cross-project learning keeps campaign evolution, reviewed knowledge, trained models, and optimizer experience separately versioned and attributable.
- Customer/project evidence is private by default, and every shared training, retrieval, seed, model, rule, or operator-statistic use is rights-authorized and consumption-traceable.
- Models, rules, and evolutionary priors declare chemistry-family, formulation-space, method, endpoint, exposure, and authority applicability and abstain or require validated adaptation outside that scope.